

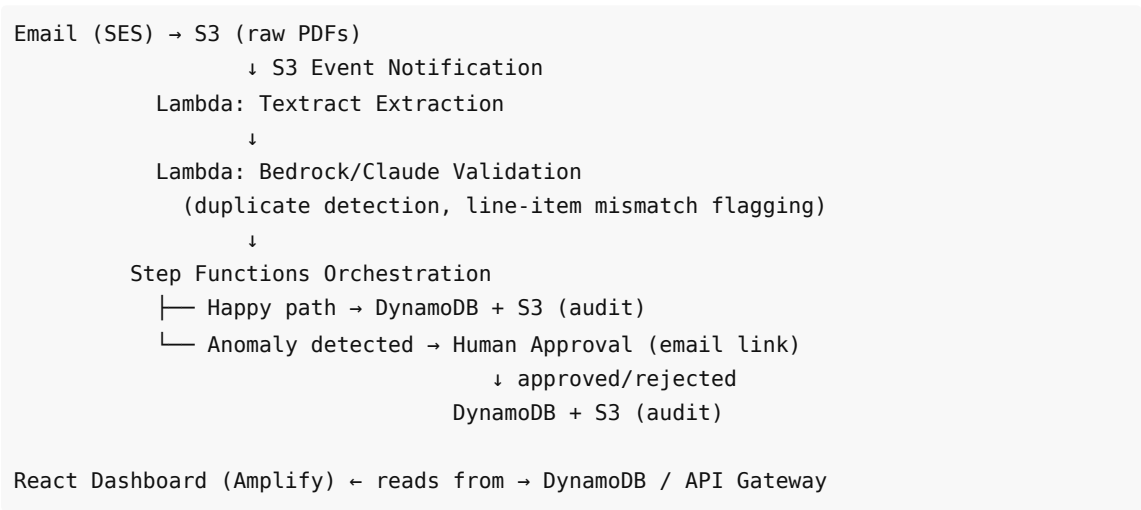
AWS Event-Driven Invoice Processing Pipeline — Project Overview

Business Context

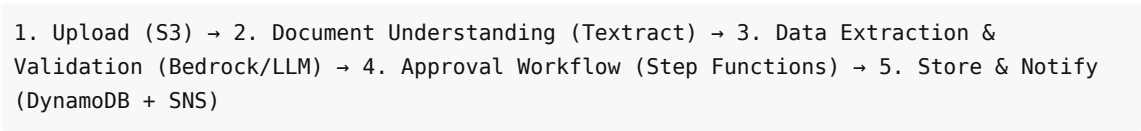
Small and medium-sized enterprises receive a high volume of invoices via email daily. Manual data entry, validation, and posting to accounting systems is slow and error-prone. This project builds an **event-driven pipeline** that:

- 1. Ingests invoice PDFs from email
- 2. Extracts fields using OCR/AI
- 3. Validates data with a large language model
- 4. Routes anomalies for human approval
- 5. Persists validated results for audit and operational use
- 6. Provides a React-based reviewer dashboard

Architecture — End-to-End Flow



5-Stage Pipeline Visualization (from UI Reference)



Key AWS Services

Service	Role
Amazon SES	Receives inbound invoices via email, stores PDF attachments
Amazon S3	Raw PDF storage, long-term audit retention of processed invoices
AWS Lambda	Event-driven compute — Textract extraction + Bedrock validation

Amazon Textract	OCR / intelligent document extraction (tables, forms, key-value pairs)
Amazon Bedrock (Claude)	LLM-based validation — duplicate detection, total vs. line-item mismatch. Also serves as fallback extraction engine when Textract is unavailable.
AWS Step Functions	Workflow orchestration — retries, timeouts, human approval branching
Amazon DynamoDB	Operational data store for processed invoices
AWS Amplify	Hosts React reviewer dashboard
AWS Secrets Manager	Stores API keys, credentials
API Gateway	REST/HTTP API for the React frontend to access backend data
Amazon SNS/SES	Sends approval/rejection emails with confirmation links

Component Breakdown

1. Ingestion Layer

- **SES Inbound Rule** → receives emails at a designated address
- **SES Action** → stores the raw email (with PDF attachment) in an S3 bucket (`s3://invoice-pipeline-raw/`)
- The PDF is extracted from the email MIME payload by a Lambda or SES directly saves the attachment
- **Manual Upload** also supported via the React frontend (drag-and-drop + browse files)

2. Document Extraction (Lambda: `textract-processor`)

- **Trigger:** Invoked by Step Functions (`ExtractDocument` state)
- **Primary:** Calls Amazon Textract `AnalyzeExpense` API for deep OCR extraction
- **Fallback:** If Textract raises `SubscriptionRequiredException` (not enabled in account), the Lambda automatically falls back to **Bedrock Converse API** (`global.anthropic.claude-sonnet-4-6`) which accepts the raw PDF/image and returns structured invoice data
- **Output:** Extracted fields (vendor name, invoice number, date, GSTIN, PO number, line items with HSN/SAC codes, quantities, unit prices, amounts, subtotal, CGST, SGST, total) → stored as JSON
- **Confidence Scores:** Per-field confidence percentage (e.g., Vendor Name 98%, Invoice Number 99%, etc.)
- **GSTIN Detection:** If the standard `TAX_PAYER_ID` field doesn't contain a GSTIN, the processor scans all extracted values for a 15-character alphanumeric Indian GSTIN pattern
- **Tax Split:** If the invoice provides a single `TAX` field, it's automatically split 50/50 into CGST and SGST (standard for Indian intra-state invoices)
- **Error handling:** Retries 3× with exponential backoff via Step Functions

3. Intelligent Validation (Lambda: `bedrock-validator`)

- **Input:** Extracted JSON from Textract (or Bedrock fallback)
- **Model:** Amazon Bedrock — `global.anthropic.claude-sonnet-4-6` (configurable via `BedrockModelId` SAM parameter)
- **Action:** Two-phase validation:
 1. **Rule-based checks** (run first, in code):

- Missing/unknown vendor name → `VENDOR_NOT_FOUND`
- Duplicate invoice number (DynamoDB query on `invoiceNumber-index` GSI) → `DUPLICATE_INVOICE`
- Missing or malformed GSTIN (validated against 15-char regex: `^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[A-Z0-9]{1}Z[A-Z0-9]{1}$`) → `MISSING_GSTIN`
- Amount mismatch: sum of line items vs. stated subtotal/total (1% tolerance, >₹1 absolute threshold) → `AMOUNT_MISMATCH`
- Extraction confidence below threshold (default 85%) → `LOW_CONFIDENCE_SCORE`

2. AI validation (Claude prompt):

- Field completeness and reasonableness
 - Vendor legitimacy assessment
 - Mathematical consistency (line × qty = amount, sum = subtotal, subtotal + tax = total)
 - Date formatting and reasonableness
 - Returns additional anomalies + per-field validation statuses (`MATCHED` , `MISMATCH` , `NOT_FOUND`)
- **Output:** Validation result with `status` (`PROCESSED` / `IN_REVIEW` / `EXCEPTION`) and `anomalies[]`
 - **Status determination:** 0 anomalies → `PROCESSED` , any HIGH severity → `EXCEPTION` , otherwise → `IN_REVIEW`
 - **Graceful degradation:** If Bedrock invocation fails, only rule-based anomalies are used

4. Orchestration (Step Functions)

- **State Machine Name:** `InvoicePipelineStateMachine-{env}`
- **Type:** `STANDARD` (for long-running human approval waits)
- **States** (from `invoice-pipeline.asl.json`):
 - `ExtractDocument` → Task: invokes `textextract-processor` Lambda (timeout: 120s)
 - `ValidateDocument` → Task: invokes `bedrock-validator` Lambda (timeout: 60s)
 - `CheckAnomalies` → Choice: branches on `$.validation.status`
 - `PROCESSED` → `PersistResults` (happy path)
 - `EXCEPTION` or `IN_REVIEW` → `HumanApproval`
 - Default → `HumanApproval`
 - `HumanApproval` → Task (`.waitForTaskToken`): invokes `approval-handler` with `SEND_APPROVAL_EMAIL` action; **pauses execution** until `SendTaskSuccess` or `SendTaskFailure` is called
 - `ProcessApprovalResult` → Choice: branches on `$.approvalResult.action`
 - `APPROVE` → `PersistResults`
 - `REPROCESS` → `ExtractDocument` (loops back)
 - Default → `HandleRejection`
 - `PersistResults` → Task: invokes `audit-logger` with `PERSIST_RESULTS` action
 - `NotifyComplete` → Task: publishes to SNS topic
 - `PipelineComplete` → Succeed
- **Error handling states:** `HandleExtractionError` , `HandleValidationError` , `HandleApprovalError` , `HandlePersistenceError` — each logs via `audit-logger` then transitions to `PipelineFailed` (Fail state)
- `HandleRejection` → logs rejection via `audit-logger` → `PipelineRejected` (Fail with error `InvoiceRejected`)
- `HandleApprovalTimeout` → logs timeout → `PipelineTimedOut` (Fail with error `ApprovalTimeout`)

- **Retry/Timeout semantics:**
 - Textract: retry 3× with exponential backoff (interval: 5s, backoff rate: 2.0)
 - Bedrock: retry 2×, timeout 60s (interval: 3s, backoff rate: 1.5)
 - Human approval: timeout 72 hours (259,200s), heartbeat every 1 hour (3,600s)
 - Persistence: retry 3× with exponential backoff

5. Human Approval / Exception Review Flow

- Step Functions sends a **Task Token** to a callback endpoint
- Email contains two links: `approve?token=xxx` and `reject?token=xxx`
- API Gateway endpoint receives the click and calls `SendTaskSuccess` / `SendTaskFailure`
- **Reviewer can also:**
 - View extracted data vs. suggested/corrected values side-by-side
 - Edit suggested corrections inline
 - Add comments (optional)
 - Choose: **Reprocess** | **Reject** | **Approve**
- Exceptions are assigned to specific reviewers
- The workflow resumes accordingly

6. Persistence & Audit

- **DynamoDB Table:** `InvoiceRecords`
 - PK: `invoiceId`, SK: `vendorId`
 - Attributes: `extractedFields`, `validationResult`, `status`, `approvedBy`, `timestamps`
- **S3 Audit Bucket:** `s3://invoice-pipeline-audit/`
 - Stores: original PDF, extracted JSON, validation report, approval metadata
 - Lifecycle policy for long-term retention

7. Reviewer Dashboard (React + Amplify)

- **Tech:** React (Vite + TypeScript), deployed on AWS Amplify
- **Design:** Dark sidebar navigation, light content area, purple/indigo accent colors
- **User Profile:** Shows user name + role (e.g., "Ananya Sharma - Finance Team")
- See **UI Reference Section** below for full page-by-page details
- **API:** Communicates with backend via API Gateway → DynamoDB

8. Secrets Management

- All credentials, API keys, Bedrock model ARNs stored in **AWS Secrets Manager**
- Lambda functions retrieve secrets at runtime (cached for performance)
- No hardcoded credentials anywhere

Infrastructure as Code (IaC)

- **AWS SAM** (primary — `template.yaml`)
- Templates cover:
 - S3 buckets (raw + audit) with bucket policies
 - Lambda functions + IAM execution roles
 - Step Functions state machine (ASL JSON)
 - DynamoDB table schema
 - API Gateway
 - SES configuration with email templates
 - Secrets Manager secrets

- Amplify hosting

Repository Structure (TCS Capstone — Final)

```
aws-invoice-pipeline/
├── backend/
│   ├── lambdas/
│   │   ├── invoice-ingestion/           # Receives SES email, extracts PDF
│   │   │   ├── handler.ts
│   │   │   ├── package.json
│   │   │   └── tsconfig.json
│   │   │   attachment, stores in S3
│   │   ├── textract-processor/         # S3 event trigger → calls Textract
│   │   │   ├── AnalyzeExpense API
│   │   │   │   ├── handler.ts
│   │   │   │   ├── package.json
│   │   │   │   └── tsconfig.json
│   │   │   ├── bedrock-validator/      # Validates extraction via Bedrock/Claude,
│   │   │   │   flags anomalies
│   │   │   │   ├── handler.ts
│   │   │   │   ├── package.json
│   │   │   │   └── tsconfig.json
│   │   │   ├── approval-handler/       # Handles approve/reject/reprocess
│   │   │   │   callbacks from email & UI
│   │   │   │   ├── handler.ts
│   │   │   │   ├── package.json
│   │   │   │   └── tsconfig.json
│   │   │   ├── audit-logger/           # Writes audit trail entries to DynamoDB +
│   │   │   │   S3
│   │   │   │   ├── handler.ts
│   │   │   │   ├── package.json
│   │   │   │   └── tsconfig.json
│   │   │   └── shared/                 # Shared code across all Lambdas
│   │   │       ├── dynamodb.ts        # DynamoDB client & helper functions
│   │   │       ├── secrets.ts          # Secrets Manager retrieval (cached)
│   │   │       ├── logger.ts           # Structured logging utility
│   │   │       └── types.ts             # Shared TypeScript interfaces/types
│   │   └── step-functions/
│   │       ├── invoice-pipeline.asl.json # Main pipeline state machine (ASL)
│   │       └── error-handling.asl.json   # Error handling sub-workflow
│   └── frontend/                       # React + Vite + TypeScript (ALREADY
│       SCAFFOLDED)
```

src/	
pages/	
Dashboard.tsx	# Stats cards, pipeline viz, chart, recent
invoices, top exceptions	
UploadInvoice.tsx	# Drag-drop upload, form, PDF preview,
extracted info preview	
InvoiceList.tsx	# Filterable/sortable table with
pagination, search, export	
InvoiceDetail.tsx	# Summary/Extraction/Approval/Audit tabs,
PDF viewer, AI validation	
Approvals.tsx	# Exception review: list + detail panel,
reprocess/reject/approve	
Analytics.tsx	# Charts, trends, processing metrics
AuditLogs.tsx	# Audit trail history per invoice
components/	
Sidebar.tsx	# Dark sidebar nav with icons, user profile
at bottom	
Header.tsx	# Page title, description, notifications
bell, user avatar	
InvoiceCard.tsx	# Invoice summary card for dashboard
StatusBadge.tsx	# Color-coded status badges (Processed/In
Review/Exception/In Progress)	
ConfidenceScore.tsx	# Circular/bar confidence indicator with
percentage	
ExceptionAlert.tsx	# Exception type badges (Missing GSTIN,
Amount Mismatch, etc.)	
PipelineStatus.tsx	# 5-stage pipeline visualization with stage
counts	
StatsCard.tsx	# Stat card with icon, value, label, trend
indicator	
InvoiceTable.tsx	# Reusable sortable/paginated data table
PDFViewer.tsx	# Embedded PDF document viewer with zoom
controls	
FilterBar.tsx	# Search + filter dropdowns (status, date
range, vendor)	
ExtractedFields.tsx	# Field name/value/confidence/validation
status table	
hooks/	
useInvoices.ts	# Fetch/filter/paginate invoices
useApprovals.ts	# Fetch pending exceptions,
approve/reject/reprocess	
useAuditLogs.ts	# Fetch audit trail data
services/	
api.ts	# Axios/fetch wrapper with base URL, auth
headers	
invoiceService.ts	# Invoice CRUD operations
approvalService.ts	# Approval workflow operations
types/	

```

├── ┌── index.ts # Shared TypeScript interfaces (Invoice,
Exception, AuditEntry, etc.)
├── ┌── App.tsx # Root component with routing
├── ┌── main.tsx # Entry point
├── ┌── index.css # Global styles
├── ┌── amplify.yml # Amplify build configuration
├── ┌── vite.config.ts # Vite config
├── ┌── tsconfig.json # TypeScript config
├── ┌── package.json # Dependencies
├── ┌── infrastructure/
├── ┌── template.yaml # AWS SAM template (main deployment)
├── ┌── s3/
├── ┌── bucket-policy.json # S3 bucket policies for raw + audit
buckets
├── ┌── dynamodb/
├── ┌── table-schema.json # DynamoDB table definition with GSIs
├── ┌── ses/
├── ┌── email-template-approval.html # Approval request email template
(approve/reject links)
├── ┌── email-template-exception.html # Exception notification email template
├── ┌── iam/
├── ┌── lambda-execution-role.json # Lambda IAM role with least-privilege
policies
├── ┌── tests/
├── ┌── unit/
├── ┌── textract.test.ts # Textract processor unit tests
├── ┌── bedrock.test.ts # Bedrock validator unit tests
├── ┌── approval.test.ts # Approval handler unit tests
├── ┌── integration/
├── ┌── pipeline.test.ts # End-to-end pipeline integration test
├── ┌── fixtures/
├── ┌── sample-invoice.pdf # Test PDF invoice
├── ┌── mock-textract-response.json # Mock Textract API response
├── ┌── docs/
├── ┌── aws_invoice_pipeline_flow.svg # Visual architecture flow diagram
├── ┌── api-reference.md # Complete REST API specification
├── ┌── api-reference.pdf # PDF version of API reference
├── ┌── deployment-guide.md # Step-by-step deployment instructions
├── ┌── deployment-guide.pdf # PDF version of deployment guide
├── ┌── overview.md # This file – detailed project overview
├── ┌── overview.pdf # PDF version of this overview
├── ┌── sample-audit-report.json # Example audit report from a processed
invoice
├── ┌── .env.example # Environment variable template
├── ┌── .gitignore # Git ignore rules
├── ┌── Makefile # Build/deploy automation commands

```

└─ README.md	# Project overview & setup instructions
└─ samconfig.toml	# SAM CLI configuration

UI Reference — Page-by-Page Specification

Design System

- **Sidebar:** Dark navy/charcoal (#1a1d2e) with purple highlight for active item
- **Content area:** White/light gray background
- **Accent color:** Purple/indigo (#6366f1) for primary actions, links, active states
- **Status colors:** Green (Processed), Orange (In Review/In Progress), Red (Exception), Blue (Resolved)
- **Typography:** Clean sans-serif (Inter or similar)
- **User profile:** Bottom of sidebar — avatar initials, name, role

Sidebar Navigation

Nav Item	Page	Icon
Dashboard	Dashboard.tsx	Grid/home icon
Upload Invoice	UploadInvoice.tsx	Cloud upload icon
Invoices	InvoiceList.tsx	Document/list icon
Processing Jobs	(optional, can merge with Dashboard)	Gear/process icon
Approval / Exceptions	Approvals.tsx	Shield/warning icon
Analytics	Analytics.tsx	Chart/bar icon
Vendors	(optional, can be a sub-page)	Building icon
Audit Logs	AuditLogs.tsx	Clock/history icon
Settings	(future)	Gear icon

Page 1: Dashboard (Dashboard.tsx)

Header: "Dashboard" — "Monitor invoice processing pipeline and system performance"

Stats Row (5 cards):

Stat	Example Value	Subtext	Color
Total Invoices	1,246	↑ 12.5% vs last 7 days	Blue icon
Processed	1,078	86.5% of total	Green icon
In Progress	98	7.9% of total	Orange icon
Exceptions	70	5.6% of total	Red icon
Total Value	₹ 2.45 Cr	↑ 18.3% vs last 7 days	Purple icon

Pipeline Visualization: Horizontal 5-stage flow with icons and counts per stage:

- 1. Upload (S3) — 1,246
- 2. Document Understanding (Textract) — 1,246
- 3. Data Extraction & Validation (Bedrock) — 1,178
- 4. Approval Workflow (Step Functions) — 1,108
- 5. Store & Notify (DynamoDB + SNS) — 1,078
- Pipeline Health bar: "Healthy" (green)
- "Last updated: 2 min ago" with refresh button

Invoices Over Time Chart: Line chart with 3 series (Processed, In Progress, Exceptions) over date range

Recent Invoices Table: 5 rows with columns: Invoice ID (link), Vendor, Amount, Status (badge), Received On — "View All" link

Top Exceptions Table: Columns: Reason, Count, Action (Review link)

- Vendor not found — 28
- Missing GSTIN — 16
- Amount mismatch — 12
- Low confidence score — 9
- Duplicate invoice — 5

Page 2: Upload Invoice (`UploadInvoice.tsx`)

Left Side:

1. **Upload Area:** Drag-and-drop zone with cloud icon + "Browse Files" button. Supports PDF, PNG, JPG (max 10MB). Shows uploaded file with name, size, upload status.
2. **Additional Information Form** (all optional):
 - Vendor (dropdown/searchable)
 - Invoice Date (date picker)
 - Purchase Order No. (text input)
 - Invoice Number (text input)
 - Notes (textarea)
3. **Actions:** "Clear All" (secondary) | "Upload & Process" (primary purple button)

Right Side: 3. **Document Preview:** Embedded PDF viewer with zoom controls, page navigation 4. **Extracted Information (Preview):** Shows real-time extraction preview after upload:

- Confidence Score (e.g., 92%) — top right badge
- Fields: Vendor Name, Total Amount, Invoice Number, GSTIN, Invoice Date, PO Number
- Note: "Extraction preview only. You can review and edit after processing."

Page 3: Invoice List (`InvoiceList.tsx`)

Header: "Invoices" — "View and manage all invoices"

Top Bar:

- Search: "Search by invoice ID, vendor, GSTIN, invoice number..."
- Export button | "Upload Invoice" button (purple)

Filter Row:

- Status dropdown (All Status)
- Date Range picker
- Vendor dropdown (All Vendors)
- "+ More Filters" button

Table Columns (all sortable ↑):

Column	Example
Invoice ID	INV-2025-1246 (clickable link)
Vendor	ABC Solutions Ltd.
Invoice Date	May 18, 2025
Amount	₹ 1,24,500.00
Status	Badge (Processed / In Review / Exception)
Received On	May 18, 2025 10:23 AM
Actions	View (eye icon) + more (⋮)

Pagination: "Showing 1 to 10 of 1246 results" with page buttons

Page 4: Invoice Detail (`InvoiceDetail.tsx`)

Header: "< Back to Invoices" | "Invoice Details" + Status Badge | "INV-2025-1246 · Received on May 18, 2025 10:23 AM" **Actions:** "Download" button | "More Actions" dropdown

Tabs: Summary | Extraction Data | Approval History | Audit Logs

Summary Tab — Left Column:

- **Invoice Information** card: Vendor Name, GSTIN, Invoice Date, Due Date, Invoice Number, PO Number, Invoice Type (badge), Created By, Created On
- **AI Validation** card: Confidence Score (92%) with progress bar, "High Confidence" badge, validation message

Summary Tab — Center Column:

- **Vendor Details** card: Company name, address, contact person, email, phone
- **Payment Summary** card: Total Amount (large), Subtotal, CGST (9%), SGST (9%)

Summary Tab — Right Column:

- **Invoice Document** viewer: Embedded PDF with toolbar (page nav, zoom, rotate, download, print, more), "Open in New Tab" link

Below Main Content:

- **Extracted Key Fields** table: Field Name | Extracted Value | Confidence (bar %) | Validation Status (Matched badge)

Page 5: Approval / Exception Review (`Approvals.tsx`)

Header: "Approval / Exception Review" — "Review and resolve exceptions in AI-processed invoices."

Stats Row (4 cards):

Stat	Value	Subtext
Total Exceptions	70	100% of total
Pending Review	24	34.3% of exceptions
In Progress	18	25.7% of exceptions
Resolved	28	40.0% of exceptions

Filter Row: Search + Issue Type (All) + Status (Pending Review) + Assigned To (All) + Filters button + Bulk Actions dropdown

Left — Exceptions List (paginated table):

Column	Details
Checkbox	For bulk actions
Invoice ID	Clickable link
Vendor	Company name
Issue Type	Colored badge (Missing GSTIN, Amount Mismatch, Vendor Not Found, Duplicate Invoice)
Confidence	Percentage
Assigned To	Reviewer name
Status	Badge (Pending Review / In Progress / Resolved)
Actions	View (eye icon) + more (⋮)

Right — Exception Detail Panel (slides in when a row is selected):

- Header: Invoice ID + Issue Type badge + Status badge + close (×)
- Info grid: Vendor, Extracted GSTIN, Invoice Date, AI Confidence, Invoice Amount, Assigned To
- **Issue Description:** Text explaining what's wrong
- **Extracted Data Table:** Field | Extracted Value | Suggested/Correct Value | Accept (✓) / Edit (✎) per row
- **Comments:** Optional comment textarea
- **Action Buttons:** Reprocess (outline) | Reject (red outline) | Approve (green solid)

Exception/Anomaly Types (from UI reference)

Exception Type	Badge Color	Description
Missing GSTIN	Red/orange	GSTIN field not found or empty in extraction
Amount Mismatch	Yellow/amber	Sum of line items ≠stated total
Vendor Not Found	Purple	Vendor not in known vendor database

Duplicate Invoice	Blue	Invoice number matches an existing record
Low Confidence Score	Orange	Overall Textract confidence below threshold

Invoice Statuses

Status	Value	Badge Color	Meaning
Processed	PROCESSED	Green	Successfully validated and persisted
In Progress	IN_PROGRESS	Orange/yellow	Currently being processed through pipeline
In Review	IN_REVIEW	Orange	Medium-severity anomaly — awaiting human review
Pending Review	PENDING_REVIEW	Yellow	Queued for review but not yet assigned
Exception	EXCEPTION	Red	High-severity anomaly detected, requires immediate attention
Resolved	RESOLVED	Blue/green	Exception reviewed and resolved (approved or rejected)

Expected Deliverables

1.  **Public GitHub repo** with all application code and IaC templates
2.  **Detailed architecture document** — state machine, retry semantics, approval flow
3.  **Sample audit report** from a batch of processed invoices
4.  **Recorded demo** of the complete workflow including anomaly → human approval

Learning Outcomes

- Event-driven architecture & loose coupling
- Generative AI integration into operational workflows
- AWS Step Functions state-machine design
- Human-in-the-loop approval workflows
- Secrets & credential management

Current Workspace State (Final)

- **Repository:** `/home/morpheus04/Desktop/aws-invoice-pipeline`
- **Frontend:** Fully built with React + Vite + TypeScript. Connected to API Gateway and deployed to AWS Amplify. Mock data removed and live data integrated.
- **Backend:** Fully implemented (5 Lambdas, Step Functions state machine, DynamoDB).
- **Infrastructure:** Fully deployed via AWS SAM (`template.yaml`).
- **Testing:** Unit tests and Integration tests implemented in `tests/` directory (Vitest). All tests passing. Edge cases (e.g. missing vendor, amount mismatches) handled and verified.
- **Status:** Project Complete! 🎉
- **Backend Language:** TypeScript (all Lambdas)

- **IaC Tool:** AWS SAM